

ΜΥΥ601 Λειτουργικά Συστήματα

Εαρινό εξάμηνο 2026

Διδάσκων: Σ. Αναστασιάδης

Εργαστήριο 1:

Υλοποίηση πολυνηματικής λειτουργίας σε μηχανή
αποθήκευσης δεδομένων

Αναφορά των φοιτητών:

Γεώργιος Γιαννούλης 5642

Οδυσσέας Γιαρένης 5632

Άγγελος-Ιορδάνης Κάκος 5572

1ο ερώτημα:

Στην υπάρχουσα υλοποίηση εντοπίστηκαν τα δύο νήματα:

Ένα νήμα υπεύθυνο για την εκτέλεση των λειτουργιών add και get. Με χρήση της εντολής: `grep -rn "pthread_create"`, βρίσκουμε τη συνάρτηση της βιβλιοθήκης pthread που δημιουργεί το δεύτερο νήμα.

```
myy601@myy601lab1:~/kiwi/kiwi-source$ grep -rn "pthread_create"
grep: engine/libindexer.a: binary file matches
grep: engine/sst.o: binary file matches
engine/sst.c:404: pthread_create(&self->merge_thread, NULL, (void (*)(void *))merge_thread, self);
```

Χρησιμοποιώντας το gdb, βάζουμε breakpoint στο σημείο που βρήκαμε την pthread_create και τρέχουμε τον debugger. Στο breakpoint τρέχουμε “info threads” και βλέπουμε πως υπάρχει 1 νήμα.

```
(gdb) info threads
Id Target Id Frame
* 1 Thread 0x7ffff7cc7740 (LWP 3725) "kiwi-bench" pthread_create_2.1 (newthread=newthread@entry=0x55555557dca0, attr=attr@entry=0x0, start_routine=start_routine@entry=0x55555555ad40 <merge_thread>, arg=arg@entry=0x55555557dab0) at ./nptl/pthread_create.c:621
```

Χρησιμοποιούμε την εντολή “bt”, η οποία μας δείχνει τη στοίβα εκτέλεσης, για να δούμε το μονοπάτι του συγκεκριμένου νήματος.

```
(gdb) bt
#0 pthread_create_2.1 (newthread=newthread@entry=0x55555557dca0, attr=attr@entry=0x0, start_routine=start_routine@entry=0x55555555ad40 <merge_thread>, arg=arg@entry=0x55555557dab0) at ./nptl/pthread_create.c:621
#1 0x000055555555942a in sst_new (basedir=basedir@entry=0x55555555631c0 "testdb", cache_size=cache_size@entry=96468992) at sst.c:404
#2 0x000055555555739c in db_open_ex (basedir=0x55555555631c0 "testdb", cache_size=96468992) at db.c:16
#3 0x00005555555560ee in write_test (count=1000, r=0) at kiwi.c:24
#4 0x00005555555560c0 in main (argc=3, argv=0x7fffffff118) at bench.c:89
```

Συνεχίζοντας την εκτέλεση του προγράμματος, ξαναχρησιμοποιούμε την εντολή “info threads” και βλέπουμε ότι πλέον υπάρχουν 2 νήματα.

```
[New Thread 0x7ffff79ff6c0 (LWP 3740)]
62 in ./sysdeps/unix/sysv/linux/x86_64/clone3.5
(gdb) info threads
Id Target Id Frame
* 1 Thread 0x7ffff7cc7740 (LWP 3725) "kiwi-bench" clone3 () at ./sysdeps/unix/sysv/linux/x86_64/clone3.5:62
2 Thread 0x7ffff79ff6c0 (LWP 3740) "kiwi-bench" futex_abstimed_wait common64 (private=0, cancel=true, abstime=0x0, op=393, expected=0, futex_word=0x55555557dc98) at ./nptl/futex-internal.c:57
```

Τώρα εκτελώντας την εντολή “bt” για το δεύτερο thread, βλέπουμε το μονοπάτι του.

```
(gdb) thread 2
[Switching to thread 2 (Thread 0x7ffff79ff6c0 (LWP 5824))]
#0 futex_abstimed_wait common64 (private=0, cancel=true, abstime=0x0, op=393, expected=0, futex_word=0x55555557dc98) at ./nptl/futex-internal.c:57
#1 ./nptl/futex-internal.c: No such file or directory.
(gdb) bt
#0 futex_abstimed_wait common64 (private=0, cancel=true, abstime=0x0, op=393, expected=0, futex_word=0x55555557dc98) at ./nptl/futex-internal.c:57
#1 futex_abstimed_wait common (futex_word=futex_word@entry=0x55555557dc98, expected=expected@entry=0, clockid=clockid@entry=0, abstime=abstime@entry=0x0, private=private@entry=0, cancel=cancel@entry=true) at ./nptl/futex-internal.c:87
#2 0x00007ffff7e50efb in GI_futex_abstimed_wait cancelable64 (futex_word=futex_word@entry=0x55555557dc98, expected=expected@entry=0, clockid=clockid@entry=0, abstime=abstime@entry=0x0, private=private@entry=0) at ./nptl/futex-internal.c:139
#3 0x00007ffff7e53558 in pthread_cond_wait common (abstime=0x0, clockid=0, mutex=0x55555557dc48, cond=0x55555557dc70) at ./nptl/pthread_cond_wait.c:503
#4 pthread_cond_wait (cond=cond@entry=0x55555557dc70, mutex=mutex@entry=0x55555557dc48) at ./nptl/pthread_cond_wait.c:618
#5 0x000055555555707b in merge_thread (data=0x55555557dab0) at sst.c:142
#6 0x00007ffff7e5413a in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:442
#7 0x00007ffff7e417de in clone3 () at ./sysdeps/unix/sysv/linux/x86_64/clone3.5:81
```

Το δεύτερο νήμα που εκτελείται στο παρασκήνιο είναι υπεύθυνο για τη συγχώνευση και τη σύμπτυξη των αρχείων SST. Ενεργοποιείται όταν πληρούνται συγκεκριμένες συνθήκες, όπως όταν το memtable γεμίσει ή όταν ο αριθμός/μέγεθος των αρχείων σε κάποιο επίπεδο ξεπεράσει ένα προκαθορισμένο όριο.

Με την εντολή: `grep -rn "pthread_exit"`, βλέπουμε ότι το δεύτερο νήμα που είχαμε δημιουργήσει πριν καταστρέφεται.

```
myy601@myy601lab1:~/kiwi/kiwi-source/engine$ grep -rn "pthread_exit"
grep: libindexer.a: binary file matches
grep: sst.o: binary file matches
sst.c:178: pthread_exit(0);
```

Τρέχοντας με gdb και βάζοντας breakpoint στο pthread_exit, παρατηρούμε με την εντολή “info threads” την ύπαρξη δύο νημάτων.

```
Thread 2 "kiwi-bench" hit Breakpoint 1, GI pthread_exit (value=value@entry=0x0) at ./nptl/pthread_exit.c:28
28 ./nptl/pthread_exit.c: No such file or directory.
(gdb) info threads
   Id Target Id                                     Frame
   * 1 Thread 0x7ffff7cc7740 (LWP 6767) "kiwi-bench" futex_wait (private=0, expected=2, futex_word=0x7ffff7f9fa10 <_IO_stdfile_1_lock>) at ../sysdeps/nptl/futex-internal
      p:146
   * 2 Thread 0x7ffff79ff6c0 (LWP 6770) "kiwi-bench" GI pthread_exit (value=value@entry=0x0) at ./nptl/pthread_exit.c:28
```

Παρατηρούμε ότι πριν την ολοκλήρωση του προγράμματος και του αρχικού νήματος, έχουμε την έξοδο και καταστροφή του δεύτερου νήματος.

```
[Thread 0x7ffff79ff6c0 (LWP 6770) exited]
[6767] 23 Mar 20:51:03.974 - file.c:170 Truncating file testdb/si/manifest to 80 bytes
[6767] 23 Mar 20:51:03.980 . log.c:46 Removing old log file testdb/si/0.log
+-----+
|Random-Write (done:1000): 0.553000 sec/op; 1.8 writes/sec(estimated); cost:553.000(sec);
|Inferior 1 (process 6767) exited with code 01|
```

Για να βρούμε, τώρα, τις λειτουργίες συγχρονισμού, εκτελούμε πρώτα την εντολή grep -rn “pthread_mutex_lock” && grep -rn “pthread_mutex_unlock”, η οποία μας δείχνει τις συναρτήσεις που χρησιμοποιούνται για αμοιβαίο αποκλεισμό.

```
myy601@myy601lab1:~/kiwi/kiwi-source/engine$ grep -rn "pthread_mutex_lock" && grep -rn "pthread_mutex_unlock"
grep: db.o: binary file matches
skiplist.c:43: pthread_mutex_lock(&self->lock);
skiplist.c:52: pthread_mutex_lock(&self->lock);
grep: libindexer.a: binary file matches
grep: compaction.o: binary file matches
grep: sst.o: binary file matches
compaction.c:187:// pthread_mutex_lock(&self->sst->lock);
compaction.c:303: pthread_mutex_lock(&self->sst->lock);
sst.c:121: pthread_mutex_lock(&self->cv_lock);
sst.c:138: pthread_mutex_lock(&sst->cv_lock);
sst.c:198: pthread_mutex_lock(&sst->immutable_lock);
sst.c:417: pthread_mutex_lock(&self->cv_lock);
sst.c:579: pthread_mutex_lock(&self->cv_lock);
sst.c:585: pthread_mutex_lock(&self->cv_lock);
sst.c:589: pthread_mutex_lock(&self->immutable_lock);
sst.c:638: pthread_mutex_lock(&self->lock);
sst.c:657: pthread_mutex_lock(&self->cv_lock);
sst.c:668: pthread_mutex_lock(&self->lock);
db.c:88: pthread_mutex_lock(&self->db->sst->immutable_lock);
db.c:204: pthread_mutex_lock(&self->db->sst->lock);
grep: skiplist.o: binary file matches
grep: db.o: binary file matches
skiplist.c:45: pthread_mutex_unlock(&self->lock);
skiplist.c:68: pthread_mutex_unlock(&self->lock);
grep: libindexer.a: binary file matches
grep: compaction.o: binary file matches
grep: sst.o: binary file matches
compaction.c:218:// pthread_mutex_unlock(&self->sst->lock);
compaction.c:329: pthread_mutex_unlock(&self->sst->lock);
sst.c:124: pthread_mutex_unlock(&self->cv_lock);
sst.c:177: pthread_mutex_unlock(&sst->cv_lock);
sst.c:203: pthread_mutex_unlock(&sst->immutable_lock);
sst.c:207: pthread_mutex_unlock(&sst->cv_lock);
sst.c:420: pthread_mutex_unlock(&self->cv_lock);
sst.c:583: pthread_mutex_unlock(&self->cv_lock);
sst.c:597: pthread_mutex_unlock(&self->immutable_lock);
sst.c:602: pthread_mutex_unlock(&self->cv_lock);
sst.c:648: pthread_mutex_unlock(&self->lock);
sst.c:663: pthread_mutex_unlock(&self->cv_lock);
sst.c:727: pthread_mutex_unlock(&self->lock);
sst.c:735: pthread_mutex_unlock(&self->lock);
db.c:99: pthread_mutex_unlock(&self->db->sst->immutable_lock);
db.c:239: pthread_mutex_unlock(&self->db->sst->lock);
grep: skiplist.o: binary file matches
```

Μερικές από αυτές χρησιμοποιούνται για τις μεταβλητές συνθήκης. Τις βρίσκουμε με την εντολή grep -rn “pthread_cond”.

```

myy601@myy601lab1:~/kiwi/kiwi-source/engine$ grep -rn "pthread_cond"
grep: db.o: binary file matches
sst.h:69: pthread_cond_t cv;
grep: merger.o: binary file matches
grep: libindexer.a: binary file matches
grep: compaction.o: binary file matches
grep: sst.o: binary file matches
sst.c:123: pthread_cond_signal(&self->cv);
sst.c:142: pthread_cond_wait(&sst->cv, &sst->cv_lock);
sst.c:147: rt = pthread_cond_timedwait(&sst->cv, &sst->cv_lock, &ts);
sst.c:402: pthread_cond_init(&self->cv, NULL);
sst.c:419: pthread_cond_signal(&self->cv);
sst.c:601: pthread_cond_signal(&self->cv);

```

Για να καταλάβουμε τη συμπεριφορά του συστήματος, θα τρέξουμε στο terminal τις εντολές read/write 100, read/write 10.000 και 100.000, ώστε να δούμε τις διαφορές στον χρόνο και στην μετακίνηση δεδομένων στο δίσκο.

Όταν κάνουμε write 100, παρατηρούμε ότι η διαδικασία ολοκληρώνεται στα 0.014 δευτερόλεπτα και ότι δημιουργείται στο δίσκο μόνο ένα επίπεδο με ένα αρχείο.

```

|Random-Write (done:100): 0.000145 sec/op; 6912.7 writes/sec(estimated); cost:0.014(sec);
|Random-Read (done:100, found:100): 0.000032 sec/op; 31526.6 reads /sec(estimated); cost:0.003(sec)

myy601@myy601lab1:~/kiwi/kiwi-source/bench/testdb/si$ ls
2 manifest
myy601@myy601lab1:~/kiwi/kiwi-source/bench/testdb/si$ cd 2
myy601@myy601lab1:~/kiwi/kiwi-source/bench/testdb/si/2$ ls
0.sst

```

Όταν κάνουμε write 10.000 παρατηρούμε ότι η διαδικασία ολοκληρώνεται στα 0.254 δευτερόλεπτα και ότι δημιουργούνται στο δίσκο 2 επίπεδα και 3 αρχεία.

```

|Random-Write (done:10000): 0.000025 sec/op; 39339.1 writes/sec(estimated); cost:0.254(sec);
|Random-Read (done:10000, found:10000): 0.000030 sec/op; 33276.8 reads /sec(estimated); cost:0.301(sec)

myy601@myy601lab1:~/kiwi/kiwi-source/bench/testdb/si$ ls
1 2 manifest
myy601@myy601lab1:~/kiwi/kiwi-source/bench/testdb/si$ cd 1
myy601@myy601lab1:~/kiwi/kiwi-source/bench/testdb/si/1$ ls
1.sst 2.sst
myy601@myy601lab1:~/kiwi/kiwi-source/bench/testdb/si/1$ cd .
myy601@myy601lab1:~/kiwi/kiwi-source/bench/testdb/si/1$ cd ..
myy601@myy601lab1:~/kiwi/kiwi-source/bench/testdb/si$ cd 2
myy601@myy601lab1:~/kiwi/kiwi-source/bench/testdb/si/2$ ls
0.sst

```

Όταν κάνουμε write 100.000, παρατηρούμε ότι η διαδικασία εκτελείται στα 2 δευτερόλεπτα.

```

|Random-Write (done:100000): 0.000029 sec/op; 34038.5 writes/sec(estimated); cost:2.938(sec);
|Random-Read (done:100000, found:100000): 0.000029 sec/op; 34824.8 reads /sec(estimated); cost:2.872(sec)

```

Τρέχοντας την εντολή write 10.000 παρατηρούμε ότι γίνεται compaction ανά 4117 κλειδιά.

```

[5471] 29 Mar 19:20:01.923 . db.c:53 Starting compaction of the memtable after 4117 insertions and 0 deletions
[5471] 29 Mar 19:20:01.923 . sst.c:595 IN sst_merge the REFCOUNT IS at 2
[5471] 29 Mar 19:20:01.923 . file.c:170 Truncating file testdb/si/1.log to 4195223 bytes
[5471] 29 Mar 19:20:01.923 . sst.c:165 The merge thread received a MERGE job
[5471] 29 Mar 19:20:01.923 . sst.c:166 Merging inside compaction thread

```

2ο ερώτημα:

Σημείωση: Για διευκόλυνσή, κάθε γραμμή κώδικα που προσθέσαμε έχει σχόλια στο τέλος της γραμμής της μορφής: `// NOTE:`

Παρακάτω ακολουθεί η υλοποίηση της καθολικής κλειδαριάς για αμοιβαίο αποκλεισμό read/write. Η υλοποίηση έγινε στα αρχεία db.h, db.c, sst.c, sst.h, kiwi.c και bench.h.

Κάνουμε include την POSIX βιβλιοθήκη “pthread.h” στο αρχείο db.h, δηλώνουμε mutex lock και την αρχικοποιούμε στο db.c.

```
typedef struct _db {  
    // char basedir[MAX_FILENAME];  
    char basedir[MAX_FILENAME+1];  
    SST* sst;  
    MemTable* memtable;  
    pthread_mutex_t lock; //NOTE: declaration of DB mutex lock  
} DB;
```

```
DB* db_open_ex(const char* basedir, uint64_t cache_size)  
{  
    DB* self = calloc(1, sizeof(DB));  
  
    if (!self)  
        PANIC("NULL allocation");  
    pthread_mutex_init(&self->lock, NULL); // NOTE: initializing mutex  
    strncpy(self->basedir, basedir, MAX_FILENAME);  
    self->sst = sst_new(basedir, cache_size);  
  
    Log* log = log_new(self->sst->basedir);  
    self->memtable = memtable_new(log);  
  
    return self;  
}
```

Στο αρχείο db.c, χρησιμοποιούμε την καθολική κλειδαριά στην db_add και db_get, ώστε να πετύχουμε αμοιβαίο αποκλεισμό μεταξύ των νημάτων που εκτελούν read ή write. Επίσης χρησιμοποιούμε το macros INFO, για να φαίνεται κάθε φορά πιο νήμα εκτελεί τις λειτουργίες add/get.

```

int db_add(DB* self, Variant* key, Variant* value)
{
    pthread_mutex_lock(&self->lock); // NOTE: locking mutex for write
    if (memtable_needs_compaction(self->memtable))
    {
        INFO("Starting compaction of the memtable after %d insertions and %d deletions",
            self->memtable->add_count, self->memtable->del_count);
        sst_merge(self->sst, self->memtable);
        memtable_reset(self->memtable);
    }
    int tmp; // NOTE: keeping the return value of memtable_add
    INFO("Thread %ld adding key", pthread_self()); // NOTE: reporting a thread adding a key
    tmp=memtable_add(self->memtable, key, value); // NOTE: adding the key while still in mutex
    pthread_mutex_unlock(&self->lock); // NOTE: unlocking mutex
    return tmp;
}

int db_get(DB* self, Variant* key, Variant* value)
{
    pthread_mutex_lock(&self->lock); // NOTE: locking mutex for read
    INFO("Thread %ld searching key", pthread_self()); // NOTE: reporting a thread searching a key
    if (memtable_get(self->memtable->list, key, value) == 1){
        pthread_mutex_unlock(&self->lock); // NOTE: unlocking mutex if we find key in memtable
        return 1;
    }
    int tmp;
    tmp=sst_get(self->sst, key, value); // NOTE: getting the key from SST while still in mutex
    pthread_mutex_unlock(&self->lock); // NOTE: unlocking mutex
    return tmp;
}

```

Μετά την δημιουργία του 2ου νήματος, παρατηρούμε ότι το νέο νήμα στέλνεται στην “merge_thread” και τίθεται σε αναμονή μέχρι να σπάσει η while και να δεχτεί signal.

```

static void merge_thread(void* data)
{
    SST* sst = (SST*)data;

    while (1)
    {
        pthread_mutex_lock(&sst->cv_lock);

        while (sst->merge_state == 0)
#ifdef WAITED
            pthread_cond_wait(&sst->cv, &sst->cv_lock);

```

Έπειτα το πρώτο νήμα συνεχίζει την εκτέλεσή του ως ότου να καλέσει την “sst_merge”, η οποία αλλάζει την μεταβλητή “merge_state”, ενημερώνει με signal την wait της “merge_thread” και το νήμα που είχε τεθεί σε αναμονή συνεχίζει την εκτέλεσή του, γυρίζοντας έπειτα σε κατάσταση αναμονής. Στη συνέχεια καλεί την “sst_free”, η οποία κάνει την ίδια διαδικασία και join, οπότε περιμένει το νήμα της “merge_thread” να τερματίσει. Αν κάνει write παραπάνω από 4.117 κλειδιά, καλεί και την “_schedule_compaction”.

Προκειμένου να αποτρέψουμε το νήμα της merge_thread να εκτελείται παράλληλα με το νήμα που κάνει add/get, ορίζουμε μέσα στο struct SST το mutex “mr_lock”, καθώς και την μεταβλητή συνθήκης “mr” στο αρχείο sst.h.

```

pthread_mutex_t mr_lock; // NOTE: mutex for add thread in combination with cond_wait
pthread_cond_t mr; // NOTE: conditional variable

```

Έπειτα, στο αρχείο `sst.c` και συγκεκριμένα στην συνάρτηση `“sst_merge”`, χρησιμοποιούμε την μεταβλητή συνθήκης που θέσαμε προηγουμένως μαζί με το `mutex`, καθώς και την μεταβλητή `“merge_state”` ως `flag` για το `while-loop`.

```
pthread_cond_signal(&self->cv);
pthread_mutex_unlock(&self->cv_lock);
pthread_mutex_lock(&self->mr_lock); // NOTE: locking mutex for cond wait while we wait for merge_thread to finish its job and signal back
//INFO("Thread %ld waiting for merge thread to finish", pthread_self());
while(self->merge_state!=0) // NOTE: waiting for merge_state to change to 0 by the merge_thread thread
{
    pthread_cond_wait(&self->mr, &self->mr_lock); // NOTE: waiting for merge_thread thread to signal
}
pthread_mutex_unlock(&self->mr_lock); // NOTE
```

Το νήμα που βρίσκεται στην “merge_thread” βγαίνει από την αναμονή και εκτελεί τις λειτουργίες του. Πριν ξαναμπει στην αναμονή, θέτει το “merge_state” ίσο με μηδέν και ενημερώνει την “mr”, ώστε να βγει από την αναμονή. Έτσι, έχουμε πετύχει αμοιβαίο αποκλεισμό.

```
sst->merge_state = 0;
INFO("Merge thread finished its job");
pthread_cond_signal(&sst->mr); // NOTE: Merge thread signals the main thread to continue working
pthread_mutex_unlock(&sst->cv_lock);
```

Για την σωστή υλοποίηση του 2ου ερωτήματος και τον έλεγχο του, πρέπει να εφαρμόσουμε πολυνηματισμό. Στο αρχείο `bench.h` δηλώνουμε, λοιπόν, ένα macro `NUM_THREADS` το οποίο καθορίζει το πλήθος των νημάτων, στην περίπτωση μας 10 νήματα. Επίσης, έχουμε φτιάξει `struct Arguments`, το οποίο θα μας βοηθήσει να δώσουμε ορίσματα στις συναρτήσεις που θα καλούν οι “`pthread_create`”.

```
#define NUM_THREADS (10) // NOTE: setting number of threads

#define LINE "+-----+-----+-----+-----+\n"
#define LINE1 "-----\n"

typedef struct args{ // NOTE: struct for passing arguments to wrapper functions of kiwi.c
    char *k;
    char *v;
    long int c;
    int ran;
    int tid;
    int nthreads;
    int f;
    int res;
    DB *base;
}Arguments;
```

To struct Arguments περιέχει:

- ***k:** Buffer για μεταβλητή “key” τύπου “Variant”
- ***v:** Buffer για μεταβλητή “value” τύπου “Variant”
- **c:** Count
- **ran:** R (Random keys)
- **tid:** Thread ID
- **nthreads:** Το πλήθος των threads
- **f:** Found (μόνο για “_read_test”)
- **res:** Αποτέλεσμα της αναζήτησης κλειδιού (μόνο για “_read_test”)
- ***base:** Δείκτης σε κοινό struct DB

Στην “_write_test” του kiwi.c, δηλώνουμε πίνακες για τα νήματα που θα δημιουργήσουμε καθώς και για τα Arguments. Αντικαθιστούμε την for-loop του αρχικού κώδικα με ένα for-loop που αρχικοποιεί κάθε Arguments και μετά δημιουργεί τα νήματα με χρήση της συνάρτησης “pthread_create”, η οποία παίρνει ως όρισμα το αντίστοιχο Arguments και καλεί την “threads_adding”, η οποία είναι wrapper function που υλοποιεί το αρχικό for-loop για κάθε νήμα.

```
void _write_test(long int count, int r)
{
    int i;
    double cost;
    //Variant sk, sv;
    double start,end;
    DB* db;

    pthread_t tid[NUM_THREADS]; // NOTE: declaration of NUM_THREADS threads
    Arguments argument[NUM_THREADS]; // NOTE: declaration of our Arguments struct for passing arguments of each thread to wrapper function threads_adding

    //char key[KSIZE + 1];
    //char val[VSIZE + 1];
    char sbuf[1024];

    //memset(key, 0, KSIZE + 1);
    //memset(val, 0, VSIZE + 1);
    memset(sbuf, 0, 1024);

    db = db_open(DATAS);

    start = get_uptime_sec();

    for(i=0; i<NUM_THREADS; i++) // NOTE: passing arguments for each thread in struct
    {
        argument[i].k=malloc(KSIZE+1);
        memset(argument[i].k, 0, KSIZE+1);
        argument[i].v=malloc(VSIZE+1);
        memset(argument[i].v, 0, VSIZE+1);
        argument[i].c=count;
        argument[i].ran=r;
        argument[i].base=db;
        argument[i].tid=i;
        argument[i].nthreads=NUM_THREADS;
        pthread_create(&tid[i], NULL, threads_adding, &argument[i]); // NOTE: creating NUM_THREADS threads and calling threads_adding with arguments
    }
}
```

Η συνάρτηση “threads_adding” καλείται από όλα τα νήματα και παίρνει ως όρισμα έναν γενικό δείκτη void* που δείχνει σε ένα struct τύπου Arguments, το οποίο περιέχει τα δεδομένα που χρειάζεται το thread για να εκτελεστεί. Έπειτα, ενημερώνει τις μεταβλητές του Arguments και καλεί την “db_add”.

```
void* threads_adding(void* arg) // NOTE: wrapper function for pthread_create calling the function db_add with arguments passing as a struct
{
    int j;
    Variant sk, sv;
    Arguments *args = (Arguments *)arg;
    for(j=args->tid; j<args->c; j+=args->nthreads)
    {
        if(args->ran)
            _random_key(args->k, KSIZE);
        else
            snprintf(args->k, KSIZE, "key-%d", j);
        fprintf(stderr, "%d adding %s\n", j, args->k);
        snprintf(args->v, VSIZE, "val-%d", j);

        sk.length = KSIZE;
        sk.mem = args->k;
        sv.length = VSIZE;
        sv.mem = args->v;
        db_add(args->base, &sk, &sv);
        if((j % 10000) == 0){
            fprintf(stderr, "random write finished %d ops%30s\r", j, "");
            fflush(stderr);
        }
    }
    return NULL;
}
```

Συνεχίζοντας στην “_write_test”, κάνουμε for-loop μέσα στο οποίο καλούμε την “pthread_join”, που περιμένει τον τερματισμό του κάθε νήματος που δημιουργήσαμε.

```
for(i=0; i<NUM_THREADS; i++) // NOTE: main thread waiting all threads in join to finish
{
    //free(argument[i].k);
    //free(argument[i].v);
    pthread_join(tid[i], NULL);
}
```


Παρόμοια διαδικασία κάνουμε για την υλοποίηση πολυνηματισμού στη “_read_test” στο αρχείο kiwi.c.

```
pthread_t tid[NUM_THREADS]; // NOTE: declaration of NUM_THREADS threads
Arguments argument[NUM_THREADS]; // NOTE: declaration of our Arguments struct for passing arguments of each thread to wrapper function threads_getting

db = db_open(DATAS);
start = get_ustime_sec();

for(i=0; i<NUM_THREADS; i++) // NOTE: passing arguments for each thread in struct
{
    argument[i].k=malloc(KSIZE+1);
    argument[i].c=count;
    argument[i].ran=ran;
    argument[i].base=db;
    argument[i].tid=i;
    argument[i].f=found;
    argument[i].nthreads=NUM_THREADS;
    pthread_create(&tid[i], NULL, threads_getting, &argument[i]); // NOTE: creating NUM_THREADS threads and calling threads_getting with arguments
}

for(i=0; i<NUM_THREADS; i++) // NOTE: main thread waiting for all threads to finish in join
{
    pthread_join(tid[i], NULL);
    found+=argument[i].f; // NOTE: computing the total amount of keys found
}

void* threads_getting(void* arg) // NOTE: wrapper function for pthread_create calling the db_get function with arguments passing as a struct
{
    int j;
    Variant sk,sv;
    Arguments *args = (Arguments *)arg;
    for(j=args->tid; j<args->c; j+=args->nthreads)
    {
        memset(args->k, 0, KSIZE+1);
        if(args->ran)
            _random_key(args->k, KSIZE);
        else
            snprintf(args->k, KSIZE, "key-%d", j);
        fprintf(stderr, "%d searching %s\n", j, args->k);
        sk.length = KSIZE;
        sk.mem = args->k;
        args->res = db_get(args->base, &sk, &sv);
        if(args->res){
            (args->f)++;
        } else {
            INFO("not found key#%s", sk.mem);
        }
        if((j % 10000) == 0){
            fprintf(stderr, "random read finished %d ops%30s\r", j, "");
            fflush(stderr);
        }
    }
    return NULL;
}
```

Όπως έχουμε προαναφέρει, merge γίνεται μετά από 4.117 κλειδιά. Θα τρέξουμε, λοιπόν, τις εντολές write 100 και 10.000, για να συγκρίνουμε και χρονικά αποτελέσματα.

Για write 100, παρατηρούμε ότι ο χρόνος είναι χειρότερος από πριν, πράγμα που είναι αναμενόμενο, καθώς έχουμε χρησιμοποιήσει το macro INFO και, επίσης, το νήμα που έχει καλέσει το merge περιμένει την ολοκλήρωση του “merge_thread”.

```
|Random-Write (done:100): 0.000306 sec/op; 3264.5 writes/sec(estimated); cost:0.031(sec);
|Random-Read (done:100, found:100): 0.000101 sec/op; 9922.6 reads /sec(estimated); cost:0.010(sec)
```

Για write 10.000, παρατηρούμε ότι ο χρόνος είναι χειρότερος για τους παραπάνω λόγους και επειδή τα νήματα που εκτελούν write περιμένουν το νήμα “merge_thread” σε περίπτωση που αυτό τρέχει.

```
|Random-Write (done:10000): 0.000047 sec/op; 21165.5 writes/sec(estimated); cost:0.472(sec);
|Random-Read (done:10000, found:10000): 0.000050 sec/op; 20115.5 reads /sec(estimated); cost:0.497(sec)
```

Κατά την ολοκλήρωση του write 10.000 παρατηρούμε ότι στο database έχουν απομείνει 1766, γεγονός που επαληθεύει ότι έχουμε επιτυχές merge ανά 4.117 κλειδιά, καθώς για

10.000 κλειδιά θα έχουμε δύο merge και θα έχουν απομείνει $10.000 - 8.234 = 1.766$ κλειδιά για το τελικό merge.

```
[1683] 01 Apr 17:48:46.207 . db.c:31 Closing database 1766
```

Τρέχοντας το πρόγραμμα με δύο νήματα παρατηρούμε ότι οι χρόνοι είναι βελτιωμένοι (Σημείωση: οι αρχικές ρυθμίσεις του VMware δεν έχουν αλλαχτεί, έχουμε, δηλαδή, τους αρχικούς 2 πυρήνες.)

```
|Random-Write (done:100): 0.000148 sec/op; 6762.3 writes/sec(estimated); cost:0.015(sec);
```

```
|Random-Read (done:100, found:100): 0.000056 sec/op; 17985.9 reads /sec(estimated); cost:0.006(sec)
```

```
|Random-Write (done:10000): 0.000054 sec/op; 18543.6 writes/sec(estimated); cost:0.539(sec);
```

```
|Random-Read (done:10000, found:10000): 0.000045 sec/op; 22223.9 reads /sec(estimated); cost:0.450(sec)
```

3ο ερώτημα:

Για να υλοποιήσουμε τον συγχρονισμό αναγνωστών - γραφέων, προσθέτουμε στο αρχείο db.h, στο struct DB, την κλειδαριά “wrl” τύπου “pthread_mutex_t”.

```
typedef struct _db {
//  char basedir[MAX_FILENAME];
  char basedir[MAX_FILENAME+1];
  SST* sst;
  MemTable* memtable;
  //pthread_mutex_t lock; // NOTE: declaration of DB mutex lock // used for part 2 only
  pthread_mutex_t wrl; // NOTE: declaration of mutex lock used for readers/writers
  pthread_cond_t can_read; // NOTE: declaration of conditional variable used for readers
  pthread_cond_t can_write; // NOTE: declaration of conditional variable used for writers
  int readers; // NOTE : count of readers in critical area
  int writers; // NOTE: count of writers in critical area (0-1)
} DB;
```

Προχωράμε στο αρχείο db.c και αρχικοποιούμε την “wrl”.

```
DB* db_open_ex(const char* basedir, uint64_t cache_size)
{
  DB* self = calloc(1, sizeof(DB));

  if (!self)
    PANIC("NULL allocation");
  //pthread_mutex_init(&self->lock, NULL); // NOTE: initializing mutex // used for part 2 only
  pthread_mutex_init(&self->wrl, NULL); // NOTE: initializing mutexes and conditional variables
  pthread_cond_init(&self->can_read, NULL);
  pthread_cond_init(&self->can_write, NULL);
  self->readers=0; // NOTE: initializing readers, writers at 0
  self->writers=0;
```

Αρχικά, υλοποιούμε προτεραιότητα στους αναγνώστες. Στην db_add, αρχικά ελέγχουμε αν υπάρχουν άλλοι γραφείς ή αναγνώστες σε λειτουργία. Αν υπάρχουν, τότε το συγκεκριμένο νήμα τίθεται σε αναμονή με χρήση “wait”. Όταν το νήμα ξυπνήσει θέτει τους γραφείς σε 1, που σημαίνει ότι ένας γραφέας βρίσκεται στην κρίσιμη περιοχή. Μετά θέτει τους γραφείς σε 0 και ειδοποιεί μέσω broadcast όλους τους αναγνώστες που μπορεί να βρίσκονται σε αναμονή και τον επόμενο γραφέα που, επίσης, μπορεί να βρίσκεται σε αναμονή. Έτσι διασφαλίζεται προτεραιότητα στους αναγνώστες.

```
int db_add(DB* self, Variant* key, Variant* value)
{
  // NOTE: we have created a readers priority queue, many readers-1 writer at a time
  // pthread_mutex_lock(&self->lock); // NOTE: locking mutex for write // used for part 2 only
  pthread_mutex_lock(&self->wrl); // NOTE: locking area for individual thread check
  while(self->readers > 0 || self->writers > 0) // NOTE: checking for readers/writers in queue or critical area
    pthread_cond_wait(&self->can_write, &self->wrl);
  self->writers = 1; // NOTE: 1 writer pass to critical area (ONLY)
  pthread_mutex_unlock(&self->wrl);

  if (memtable_needs_compaction(self->memtable))
  {
    INFO("Starting compaction of the memtable after %d insertions and %d deletions",
        self->memtable->add_count, self->memtable->del_count);
    sst_merge(self->sst, self->memtable);
    memtable_reset(self->memtable);
  }
  int tmp; // NOTE: keeping the return value of memtable_add
  INFO("Thread %ld adding key", pthread_self()); // NOTE: reporting a thread adding a key
  tmp=memtable_add(self->memtable, key, value); // NOTE: adding the key while still in mutex
  // pthread_mutex_unlock(&self->lock); // NOTE: unlocking mutex
  pthread_mutex_lock(&self->wrl);
  self->writers = 0; // NOTE: reporting writer has finished
  pthread_cond_broadcast(&self->can_read); // NOTE: letting readers start if any
  pthread_cond_signal(&self->can_write); // NOTE: letting the next writer start if any
  pthread_mutex_unlock(&self->wrl);
  return tmp;
}
```

Παρομοίως, στην `db_get`, ελέγχουμε για τυχόν γραφέα σε λειτουργία και, αν υπάρχει, περιμένουμε την ολοκλήρωση του και το broadcast του, ώστε να ξεκινήσουν οι αναγνώστες που βρίσκονται σε αναμονή την εκτέλεση τους. Επίσης, μετράμε το πλήθος των αναγνωστών που περνάνε στην κρίσιμη περιοχή και στο τέλος ελέγχουμε αν όλοι έχουν ολοκληρωθεί, ώστε να ειδοποιήσουμε τυχόν γραφέα σε αναμονή.

```
int db_get(DB* self, Variant* key, Variant* value)
{
    // NOTE: we have created a readers priority queue, many readers-1 writer at a time
    // pthread_mutex_lock(&self->lock); // NOTE: locking mutex for read // used for part 2 only
    pthread_mutex_lock(&self->wrl);
    while(self->writers) // NOTE: checking for writer in critical area, if not continue
        pthread_cond_wait(&self->can_read, &self->wrl);
    self->readers++; // NOTE: keeping track of readers to signal writers when done (if any)
    pthread_mutex_unlock(&self->wrl);
    INFO("Thread %ld searching key", pthread_self()); // NOTE: reporting a thread searching a key
    if (memtable_get(self->memtable->list, key, value) == 1){
        //pthread_mutex_unlock(&self->lock); // NOTE: unlocking mutex if we find key in memtable // used for part 2 only
        pthread_mutex_lock(&self->wrl);
        self->readers--; // NOTE: reader has finished
        if(!self->readers)
            pthread_cond_signal(&self->can_write); // NOTE: letting writers in queue start (if any)
        pthread_mutex_unlock(&self->wrl);
        return 1;
    }
    int tmp;
    tmp=sst_get(self->sst, key, value); // NOTE: getting the key from SST to return
    // pthread_mutex_unlock(&self->lock); // NOTE: unlocking mutex // used for part 2 only
    pthread_mutex_lock(&self->wrl);
    self->readers--; // NOTE: same logic as if key found in memtable
    if(!self->readers)
        pthread_cond_signal(&self->can_write);
    pthread_mutex_unlock(&self->wrl);
    return tmp;
}
```

Σε αυτό το στάδιο, αντικαθιστούμε στο αρχείο `sst.h` τον μηχανισμό συγχρονισμού του 2ου ερωτήματος με τον αρχικό, γιατί πλέον οδηγεί σε αδιέξοδο, γεγονός που μπορούσαμε να είχαμε υλοποιήσει εξαρχής.

```
//pthread_mutex_t mr_lock; // NOTE: mutex for add thread in combination with cond_wait // used for part 2, replaced with
//pthread_cond_t mr; // NOTE: conditional variable // original cv_lock mutex and cv con

sst->merge_state = 0;
INFO("Merge thread finished its job");
pthread_cond_signal(&sst->cv); // NOTE: Merge thread signals the main thread in sst_merge to continue working
pthread_mutex_unlock(&sst->cv_lock);

pthread_cond_signal(&self->cv);
pthread_mutex_unlock(&self->cv_lock);
pthread_mutex_lock(&self->cv_lock);
//pthread_mutex_lock(&self->mr_lock); // NOTE: locking mutex for cond_wait while we wait for merge_thread to finish its job and signal back // used for part 2,
//switched to original cv_lock
//INFO("Thread %ld waiting for merge thread to finish", pthread_self());
while(self->merge_state!=0) // NOTE: waiting for merge_state to change to 0 by the merge_thread thread
{
    pthread_cond_wait(&self->cv, &self->cv_lock); // NOTE: waiting for merge_thread thread to signal
}
pthread_mutex_unlock(&self->cv_lock);
```

Τρέχοντας τώρα το πρόγραμμα με 10 νήματα, παρατηρούμε ότι χρόνοι είναι βελτιωμένοι από όλες τις προηγούμενες υλοποιήσεις.

```
|Random-Write   (done:100): 0.000113 sec/op; 8867.6 writes/sec(estimated); cost:0.011(sec);
|Random-Read    (done:100, found:100): 0.000091 sec/op; 10984.2 reads /sec(estimated); cost:0.009(sec)
|Random-Write   (done:10000): 0.000038 sec/op; 26231.0 writes/sec(estimated); cost:0.381(sec);
|Random-Read    (done:10000, found:10000): 0.000029 sec/op; 34854.6 reads /sec(estimated); cost:0.287(sec)
```

4ο ερώτημα:

Για την επέκταση του bench.c, προσθέσαμε την δυνατότητα επιλογής readwrite στα ορίσματα γραμμής εντολών. Μπορούμε να επιλέξουμε το ποσοστό λειτουργιών add και get γράφοντας “./kiwi-bench readwrite Reads% Writes% Count”. Το πρόγραμμα επαληθεύει ότι το άθροισμα των ποσοστών είναι 100% και καλεί την συνάρτηση “_readwrite_test”. Πριν τελειώσει ενημερώνει τον χρήστη για το συνολικό νούμερο νημάτων που χρησιμοποιήθηκαν.

```
if (argc < 3) {
    fprintf(stderr, "Usage: db-bench <write | read | readwrite <%read %write> <count>\n"); // NOTE: for readwrite:
                                                    // ./kiwi-bench readwrite <%reads> <writes> <count>
    exit(1);
}

else if (strcmp(argv[1], "readwrite") == 0) // NOTE: checking for user's readwrite request
{
    if (argc != 6 && argc != 5)
    {
        fprintf(stderr, "Not enough arguments\n");
        exit(1);
    }
    int r=0;
    int writes;
    int reads;
    reads = atoi(argv[2]);
    writes = atoi(argv[3]);
    if (reads+writes != 100) // NOTE: if read-write percentages don't sum up to 100 exit
    {
        fprintf(stderr, "%read %write should be percentages\n");
        exit(1);
    }
    count = atoi(argv[4]);
    _print_header(count);
    _print_environment();
    if (argc == 6) // NOTE: checking for random keys request
        r = 1;
    _readwrite_test(count, reads, r); // NOTE: calling a new function in kiwi.c
}
else {
    fprintf(stderr, "Usage: db-bench <write | read | readwrite <%read %write> <count> <random>\n");
    exit(1);
}
printf("\nNOTE: For this execution we used %d threads\n", NUM_THREADS); // NOTE: before exiting informing the user about
                                                                    // the total of threads used for execution of kiwi
return 1;
```

Στο αρχείο bench.h, κάνουμε αλλαγές στο struct Arguments, ώστε να μπορούμε να το χρησιμοποιήσουμε και για την συνάρτηση “threads_readwrite”. Συγκεκριμένα, προσθέσαμε:

- **wr:** Πλήθος εγγραφών κάθε νήματος
- **per_read:** Ποσοστό reads
- **comprand:** Αριθμός που υπολογίζεται με χρήση rand() και καθορίζει με βάση το “per_read” τι λειτουργία θα εκτελεστεί
- **searches:** Πλήθος αναζητήσεων κάθε νήματος (πετυχημένες και μη)

```
typedef struct args{ // NOTE: struct for passing arguments to wrapper functions of kiwi.c
    char *k;
    char *v;
    long int c;
    int ran;
    int tid;
    int nthreads;
    int f;
    int wr;
    int res;
    int per_read;
    int comprand;
    int searches;
    DB *base;
}Arguments;
```

Συνεχίζουμε στο αρχείο kiwi.c με την υλοποίηση της συνάρτησης “_readwrite_test”. Μέσω μιας for-loop αρχικοποιούμε κάθε Arguments και δημιουργούμε νήματα, τα οποία παίρνουν ως όρισμα τα αντίστοιχα Arguments και ξεκινούν να τρέχουν την wrapper function “threads_readwrite”. Έπειτα, το κύριο νήμα περιμένει το τερματισμό κάθε νήματος με την χρήση της “pthread_join” και υπολογίζει το συνολικό πλήθος των εγγραφών, αναζητήσεων και αναγνώσεων, προσθέτοντας τις αντίστοιχες μεταβλητές των Arguments κάθε νήματος.

```
void _readwrite_test(long int count, int reads, int r) // NOTE: function used for readwrite request
// NOTE: logic same as in _read_test, _write_test
{
    int i, found=0, written=0, searches=0; // NOTE: counting total keys written, searched and found
    double cost;
    double start, end;
    DB* db;

    pthread_t tid[NUM_THREADS];
    Arguments argument[NUM_THREADS];

    char sbuf[1024];
    memset(sbuf, 0, 1024);

    db = db_open(DATAS);

    start = get_ustime_sec();

    for(i=0; i<NUM_THREADS; i++) // NOTE: passing arguments for each thread in struct
    {
        argument[i].k=malloc(KSIZE+1);
        memset(argument[i].k, 0, KSIZE+1);
        argument[i].v=malloc(VSIZE+1);
        memset(argument[i].v, 0, VSIZE+1);
        argument[i].c=count;
        argument[i].ran=r;
        argument[i].base=db;
        argument[i].tid=i;
        argument[i].nthreads=NUM_THREADS;
        argument[i].f=found;
        argument[i].wr=written;
        argument[i].per_read=reads;
        argument[i].searches=searches;
        pthread_create(&tid[i], NULL, threads_readwrite, &argument[i]);
    }

    for(i=0; i<NUM_THREADS; i++) // NOTE: main thread waiting for each thread to finish in join
    {
        pthread_join(tid[i], NULL);
        found+=argument[i].f; // NOTE: computing total amount of keys found
        written+=argument[i].wr; // total amount of keys written
        searches+=argument[i].searches; // total amount of keys searched
        free(argument[i].k);
        free(argument[i].v);
    }

    db_close(db);

    end = get_ustime_sec();
    cost = end - start;
    printf(LINE);
    printf("Readwrite (written:%d, read:%d, found:%d): %.6f sec/op; %.1f reads /sec(estimated); cost:%.3f(sec)\n", written, searches, found, (double)(cost / count), (double)(count / cost), cost);
}
```

Η συνάρτηση “threads_readwrite” εκτελείται από κάθε νήμα και μέσω γεννήτριας τυχαίων αριθμών, από το 0 μέχρι το 99, αποφασίζει αν θα εκτελέσει read ή write συγκρίνοντας τον τυχαίο αριθμό με την μεταβλητή ποσοστού reads που έχουμε περάσει ως όρισμα στα Arguments. Παράλληλα, η συνάρτηση ενημερώνει τα στατιστικά στοιχεία κάθε νήματος.

```

void* threads_readwrite(void* arg) // NOTE: wrapper function for pthread_create calling either db_add or db_get depending
// on a possibility combined with user's % of reads request, all arguments passed as a struct
{
    int j;
    Variant sk,sv;
    Arguments *args = (Arguments *)arg;
    for(j=args->tid; j<args->c; j+=args->nthreads)
    {
        args->comprand = rand() % 100; // NOTE: getting a number between 0-99 as a possibility
        if(args->per_read > args->comprand) // NOTE: if %reads > possibility, we have a reader
        {
            // READ
            memset(args->k, 0, KSIZE+1);
            if(args->ran)
                _random_key(args->k, KSIZE);
            else
                snprintf(args->k, KSIZE, "key-%d", j);
            fprintf(stderr, "%d searching %s\n", j, args->k);
            sk.length = KSIZE;
            sk.mem = args->k;
            args->res = db_get(args->base, &sk, &sv);
            args->searches++; // NOTE: counting each thread's searches even if key not found
            if(args->res)
                args->f++;
            else
                INFO("not found key%s", sk.mem);
            if((j % 10000) == 0){
                fprintf(stderr, "random read finished %d ops%30s\r", j, "");
                fflush(stderr);
            }
        }
        else
            // WRITE

```

```

        {
            if(args->ran)
                _random_key(args->k, KSIZE);
            else
                snprintf(args->k, KSIZE, "key-%d", j);
            fprintf(stderr, "%d searching %s\n", j, args->k);
            sk.length = KSIZE;
            sk.mem = args->k;
            sv.length = VSIZE;
            sv.mem = args->v;
            db_add(args->base, &sk, &sv);
            args->wr++;
            if((j % 10000) == 0){
                fprintf(stderr, "random read finished %d ops%30s\r", j, "");
                fflush(stderr);
            }
        }
    }
    return NULL;
}

```

Παρατηρήσαμε ότι αν έχουμε εγγραφές από προηγούμενη εκτέλεση η εντολή readwrite, με μη μηδενικά ποσοστά, λαμβάνει σήμα SIGABRT και τερματίζει χωρίς να ολοκληρωθεί.

Thread 7 "kiwi-bench" received signal SIGABRT, Aborted.

(Εικόνα από gdb)

**corrupted size vs. prev_size
Aborted**

(Εικόνα από μήνυμα τερματικού)

Αυτό το πρόβλημα δεν εμφανίζεται αν υπάρχει μηδενικό read ή write, όπως θα εξηγηθεί παρακάτω.

Θα τρέξουμε, τώρα, την readwrite για 10 νήματα. Η εντολή “readwrite 50 50 100” σημαίνει ότι από 100 κλειδιά περίπου το 50% θα κάνει read και το άλλο 50% write. Στη συγκεκριμένη περίπτωση, τυχαίνει να έχουν μοιραστεί τα νήματα ακριβώς, αλλά αυτό δεν ισχύει πάντα. Επίσης, έχουμε found 0, καθώς δεν είχαμε εγγραφές πριν την εκτέλεση, και έτσι, κάθε κλειδί

που αναζητήθηκε ήταν μοναδικό και δεν υπήρχε εγγραφή για αυτό από άλλο νήμα. Γενικά, κάθε κλειδί ή διαβάζεται ή γράφεται, οπότε από την στιγμή που δεν έχουμε εγγραφές από προηγούμενη εκτέλεση, τα κλειδιά που θα αναζητηθούν δεν υπάρχουν, άρα, εν τέλει, το found θα είναι 0 και ας έχουν ολοκληρωθεί αναγνώσεις (read). Τρέχουμε, επίσης, την εντολή "readwrite 50 50 10.000" και παρατηρούμε το ίδιο.

```
Readwrite (written:50, read:50, found:0): 0.000207 sec/op; 4829.7 reads /sec(estimated); cost:0.021(sec)
```

```
Readwrite (written:5008, read:4992, found:0): 0.000063 sec/op; 15752.1 reads /sec(estimated); cost:0.635(sec)
```

Σχετικά με το προηγούμενο πρόβλημα, αν έχουμε από προηγούμενη εκτέλεση 513 εγγραφές και εκτελέσουμε "readwrite 100 0 1.000", σαν να κάνουμε, δηλαδή, "read 1.000", παρατηρούμε ότι η εκτέλεση ολοκληρώνεται κανονικά και βρίσκει και τα 513 κλειδιά, χωρίς να κάνει καινούργιες εγγραφές.

```
Readwrite (written:0, read:1000, found:513): 0.000042 sec/op; 24055.7 reads /sec(estimated); cost:0.042(sec)
```

Αντίστοιχα, αν τρέξουμε "readwrite 0 100 1.000", ουσιαστικά εκτελείται "write 1.000" και γράφονται και τα 1.000 κλειδιά, είτε έχουμε προηγούμενες εγγραφές στον δίσκο, είτε όχι, χωρίς να γίνονται αναγνώσεις.

```
Readwrite (written:1000, read:0, found:0): 0.000048 sec/op; 21005.3 reads /sec(estimated); cost:0.048(sec)
```

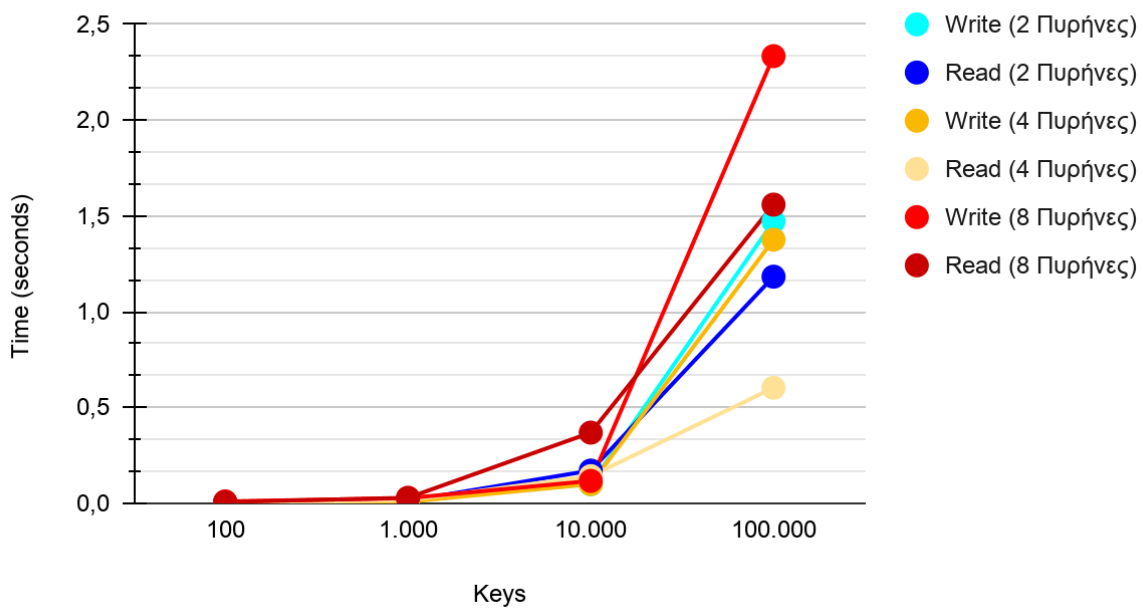
Σημείωση: Για να επαληθεύσουμε ότι δεν έχουμε απώλεια κλειδιών, ελέγχουμε ότι το άθροισμα των written και των read ισούται με το Count.

5ο ερώτημα:

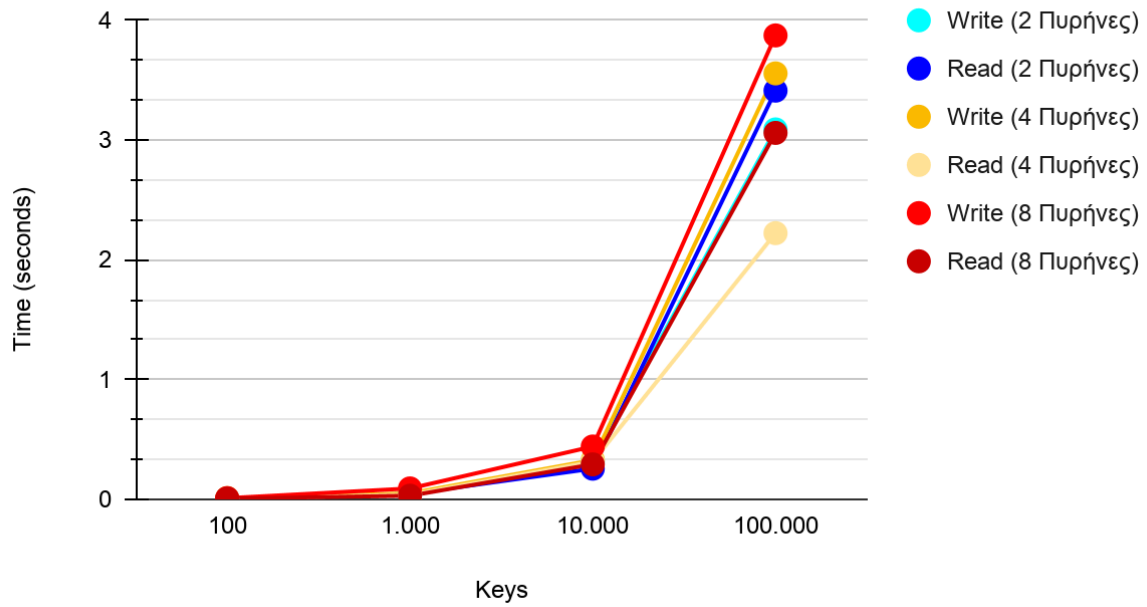
Ακολουθούν τα γραφήματα τύπου Χρόνου - Κλειδιών (Time - Keys) για τα διάφορα νηματα και για τις λειτουργίες read και write. Οι μετρήσεις έγιναν με τις παρακάτω παραμέτρους:

- 100, 1.000, 10.000, 100.000 κλειδιά
- Αρχικό κώδικα και πολυνηματισμό με 2, 4, 8 νήματα
- 2, 4, 8 πυρήνες

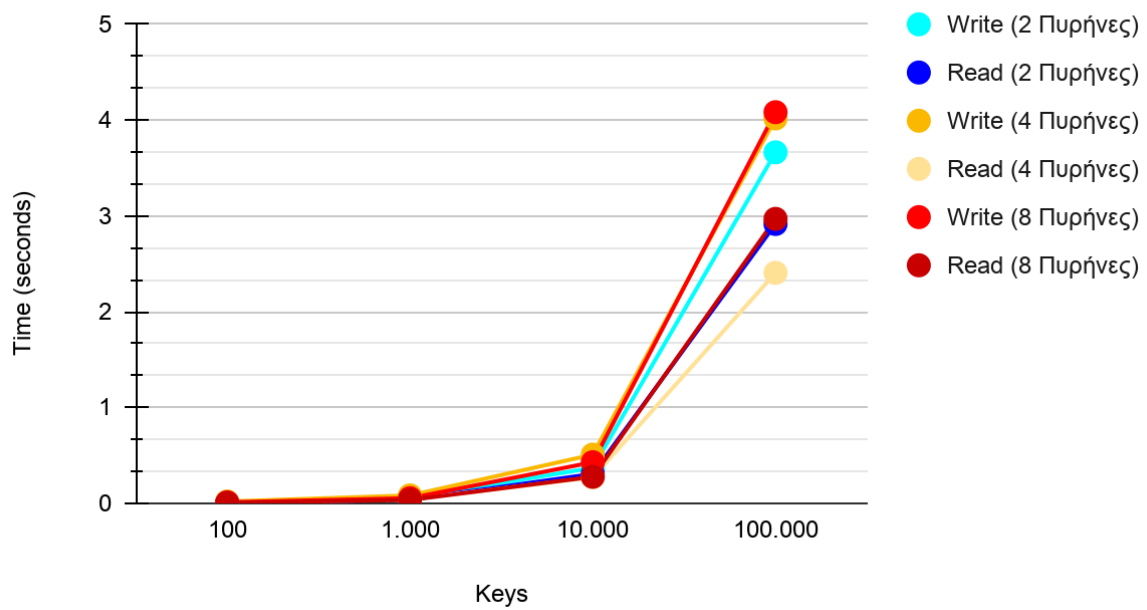
Αρχικός Κώδικας



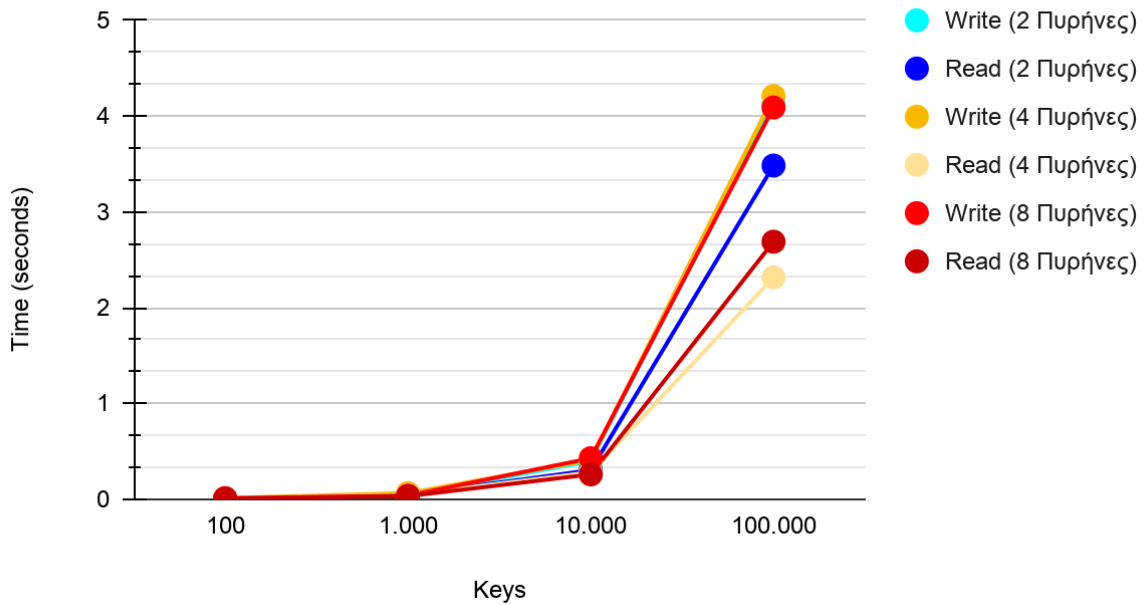
2 Νήματα



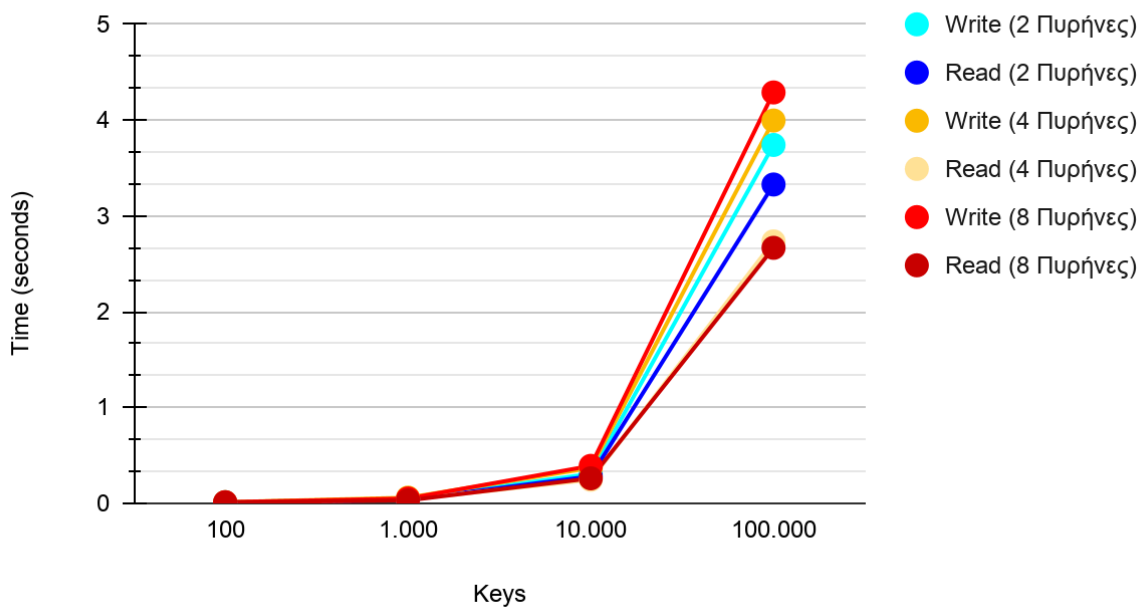
4 Νήματα



6 Νήματα



8 Νήματα



Παρατηρήσεις

Είναι αναμενόμενο ότι γενικότερα οι write θα είναι πλέον σχετικά πιο αργές σε σχέση με τον αρχικό κώδικα, καθώς τα merges δεν συμβαίνουν παράλληλα με τα writes, τα οποία τίθενται σε αναμονή μέχρι την ολοκλήρωση του merge. Αντιθέτως, τα reads δεν έχουν αυτό το θέμα,

αφού δεν συμβαίνουν merges, κάνοντας έτσι την μηχανή μας σχετικά γρηγορότερη για αυτή την λειτουργία. Επίσης, παρατηρούμε ότι στον αρχικό κώδικα η αύξηση των επεξεργαστικών πυρήνων συνεπάγεται, συνήθως, και σε αύξηση ταχύτητας, ιδίως σε μεγάλο πλήθος κλειδιών. Παρατηρείται, επίσης, η αύξηση της ταχύτητας της μηχανής μας παράλληλα με την αύξηση των πυρήνων για μικρό πλήθος νημάτων, αφού και η δημιουργία, διαχείριση και καταστροφή των νημάτων αποτελεί από μόνη της μια χρονοβόρα διαδικασία. Αυτό ισχύει γενικά για τα μικρότερα πλήθη κλειδιών, καθώς η χρήση πολλών νημάτων είναι πιο αποδοτική στα μεγάλα πλήθη κλειδιών, επικαλύπτοντας έτσι την χρονική καθυστέρηση που αναφέραμε. Γενικά η αύξηση του πλήθους των πυρήνων παράλληλα με την αύξηση του πλήθους των νημάτων είναι το ιδανικό σενάριο, όταν ασχολούμαστε με μεγάλο πλήθος δεδομένων.

Τελικές σημειώσεις

Σας παραθέτουμε και την έξοδο της τελικής make.

```
cd engine && make all
make[1]: Entering directory '/home/myy601/kiwi/kiwi-source/engine'
make[1]: Nothing to be done for 'all'.
make[1]: Leaving directory '/home/myy601/kiwi/kiwi-source/engine'
cd bench && make all
make[1]: Entering directory '/home/myy601/kiwi/kiwi-source/bench'
gcc -g -ggdb -Wall -Wno-implicit-function-declaration -Wno-unused-but-set-variable bench.c kiwi.c -L ../engine -lindexer -lpthread -lsnappy -o kiwi-bench
make[1]: Leaving directory '/home/myy601/kiwi/kiwi-source/bench'
```

Για να έχουμε ακρίβεια στους χρόνους, αλλάξαμε τα παρακάτω αρχεία:

- util.c
- util.h
- bench.h
- kiwi.c